Database Recovery

The objective is to ensure that database transactions satisfy the ACID properties, particularly:

- **Atomicity**: A transaction must either complete entirely or have no effect.
- **Durability**: Once a transaction commits, its changes must survive failures.

Let's explore the major database recovery techniques.

Types of Failures

- **Computer failure**: A hardware, software or network error causes the system to crash.
- **A transaction or system error**: An operation in the transaction may cause it to fail if inaccurate input has been supplied or because of a programming error. The user may interrupt the transaction during its execution.
- **Local errors or exception conditions detected by the transaction**: during the transaction something may happen that necessitates cancellation of the transaction e.g. data for the transaction may not be found or, for example, a bank balance may not have enough funds to complete the transaction.
- **Concurrency control enforcement**: Two workstations may try to update the database at the same time.
- **Catastrophic failures**: Physical problems like Disk failure.

Different recovery techniques address different types of failures.

Types of Recovery Techniques

1. Log based Recovery
2. Shadow Paging
3. Backup and Restore

For **Catastrophic failures** Backup and Restore is only option. May be keep log on different storage, or so

Some Terms

In-place updating: Modified buffer is written at same original disk location thus overwriting the old value of changed data item.

Shadow update: The modified version of a data item does not overwrite its disk copy but is written at a separate disk location

Log-based recovery

This is one of the most fundamental database recovery techniques.

To recover from failures that affect transactions, the **system maintains a log to keep track of all transaction operations that affect the values of database items**.

Recovery manager keeps track of the following operations of transactions-

- Begin Transaction

- Read or Write
- End Transaction operation by the transaction: i.e. Commit or Rollback

Typically (dependents on recovery technique), following entries called log records, are placed in log-

```
[start_transaction, T]
    -- indicates start of transaction T, where T is a unique ID for the transaction
[write, T, X, OldValue, NewValue]
    -- T has changed X from old-value to new value
[read, T, X]
[commit, T] or [abort, T]
```

For example, suppose we have:

```
Account A = 1000
Account B = 2000
```

A transaction transfers 500 from A to B.

```
BEGIN TRANSACTION;

UPDATE Account
SET balance = balance - 500
WHERE account_id = 'A';

UPDATE Account
SET balance = balance + 500
WHERE account_id = 'B';

COMMIT;
```

The transaction log might contain:

Log record	Description
`<START T1>`	Transaction begins
`<T1, A, 1000, 500>`	A changes from 1000 to 500
`<T1, B, 2000, 2500>`	B changes from 2000 to 2500
`<COMMIT T1>`	Transaction commits

The log stores the old and new values.

There are two important operations during recovery:

- **UNDO**: Reverse the changes made by an incomplete transaction.
- **REDO**: Reapply changes made by a committed transaction if they were not yet persisted.

Have Deferred database UPDATE

The deferred update technique does not **physically update the database on the disk, until the transaction commits**, till then updates are applied to local transaction's work space.

During Commit, the updates are first recorded in log file and then written the database.
If transaction fails before commit no undoing is required, however there is need of redoing of committed transactions from the log. This type of recovery technique is called **Non-Undo/ReDo algorithm**.

Consider:

```
T1 starts
T1 updates A
T1 updates B
T1 commits
```

Suppose the system crashes immediately after the commit record is written but before the updated data reaches the database files.

During recovery:

- T1 is committed.
- Its changes must be applied again.
- Therefore, the DBMS performs REDO.

An uncommitted transaction does not need UNDO because its modifications were never applied to the database.

Immediate database UPDATES

Database is updated before transaction reaches to commit point.

However these operations are recorded in the log on disk by force before they are applied to the database, making recovery still possible.

If a transaction fails after recording some changes but before reaching commit, its operations need to be undone, i.e. the transaction need to be rolled back. In immediate update **both undoing and redoing might have to be done for recovery purpose**, that is why it is called as **UnDo/ReDo algorithm**

For example:

```
T1 starts
T1 updates A
T1 updates B
T1 commits
```

Suppose the system crashes after updating A but before updating B.

There are two possibilities:

- If T1 had committed, REDO its changes as necessary.
- If T1 had not committed, UNDO its changes.

Recovery: UNDO + REDO

Checkpointing

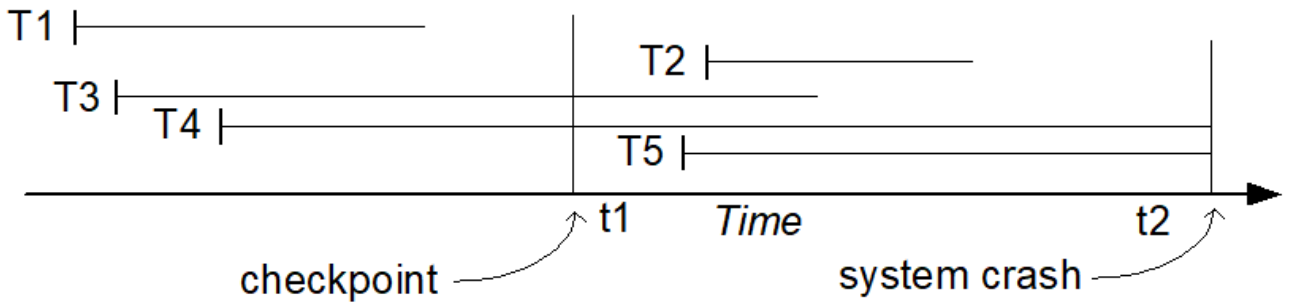
Imagine a database has been running for several days and has generated millions of log records.

If the system crashes, scanning the entire log from the beginning would be expensive.

A checkpoint reduces the amount of work required during recovery.

At certain intervals, the DBMS creates a checkpoint.

Consider the figure below. A check point was inserted at time t1, and the system crashed at time t2.



Transaction T1 was committed before check point, no need to be considered by recovery
T2 and T3 was committed before crash, therefore **need to be redone**.
T4 and T5 was not committed, therefore **require undoing**. (assuming immediate update)

So Checkpointing helps in making recovery efficient.

Important: A checkpoint does not necessarily mean that every dirty page has been written to disk. That depends on the database update protocol.

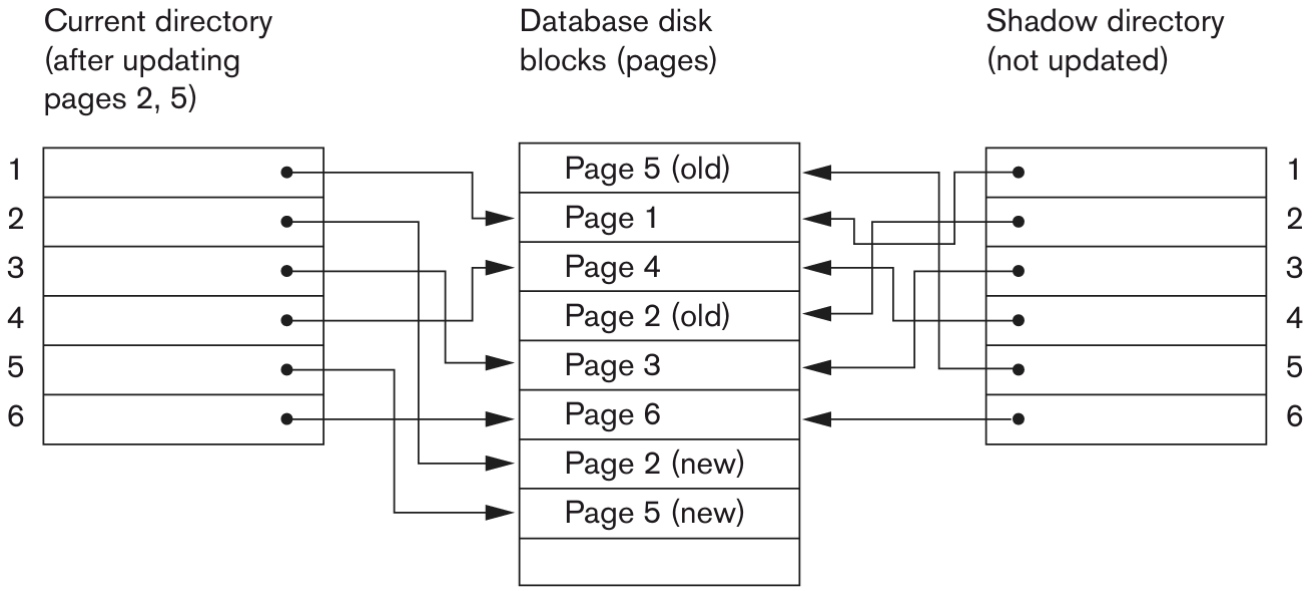
Shadow Paging

Shadow paging is an alternative to traditional log-based recovery.

The basic idea is to maintain two page tables:

1. Shadow page table: Points to the original database pages.
2. Current page table: Points to modified pages.

When a transaction modifies a page, the DBMS creates a new copy rather than overwriting the original page.



When the transaction commits:

- The DBMS atomically switches the database root pointer to the new page table.
- The new version becomes the committed database state.

If a crash happens before the switch, the old shadow page table remains valid.

Advantages:

- No conventional UNDO/REDO log is required for page recovery.
- Recovery can be relatively simple.

Disadvantages:

- Copying pages and managing page tables introduces overhead.
- Fragmentation and garbage collection can become problems.
- It is difficult to use efficiently with high concurrency.

ARIES Recovery Algorithm

Important in database internals

ARIES stands for: **Algorithms for Recovery and Isolation Exploiting Semantics**.

It is a widely studied recovery algorithm, associated with IBM research and used as a foundation for understanding recovery in modern storage engines.

ARIES uses:

- Write-Ahead Logging (WAL)
- UNDO
- REDO
- Checkpoints
- Log Sequence Numbers (LSNs)

ARIES recovery has three phases:

Phase 1: Analysis

Determine which transactions were active at the time of the crash and which database pages might need recovery.

Phase 2: REDO

Repeat history by reapplying necessary logged changes to reconstruct the database state at the moment of the crash.

Phase 3: UNDO

Reverse the effects of transactions that had not committed before the crash.

One important feature of ARIES is that it uses Compensation Log Records (CLRs) to record undo operations, allowing recovery itself to be restarted safely if another crash occurs.

Summary: Comparison of recovery techniques

Technique	Main idea	Recovery operation
Deferred modification	Apply changes after commit	REDO
Immediate modification	Apply changes before commit	UNDO + REDO
Checkpointing	Reduce recovery work	Supports faster recovery
Shadow paging	Maintain old and new page tables	Pointer restoration
ARIES	WAL-based three-phase recovery	Analysis + REDO + UNDO
Backup and restore	Restore persistent database copies	Restore + log replay